LAPORAN PRAKTIKUM PERTEMUAN 10
KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK

LOGGING TERSTRUKTUR, ENDPOINT /HEALTH, DAN HEALTHCHECK REAKTIF DUA ARAH PADA DOCKER COMPOSE

Nama	:	Erico Ali Gutama
NIM	:	24/542626/SV/25066
Kelas	:	B1
Dosen Pengampu	:	Galih Malela Damaraji

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

BAB I. PENGALIHAN LOG LARAVEL KE STDOUT (SOAL 1)

1.1 Prinsip Desain Log sebagai Event Streams (12-Factor App):

Dalam rekayasa sistem terdistribusi modern dan ekosistem kontainer, prinsip *The Twelve-Factor App* butir XI (*Logs as Event Streams*) menyatakan bahwa aplikasi tidak boleh membebankan dirinya dengan urusan pengelolaan atau penyimpanan berkas log pada sistem berkas lokal. Kontainer bersifat sementara (*ephemeral*), sehingga penulisan berkas log lokal di dalam kontainer akan menyulitkan agregasi log terpusat dan berisiko memenuhi kapasitas penyimpanan kontainer secara liar. Oleh karena itu, seluruh luaran log dialihkan secara langsung ke standar output (stdout), di mana runtime kontainer (Docker Engine) dapat menangkap, merotasi, dan menyalurkannya ke agregator log.

1.2 Implementasi Konfigurasi pada Repositori:

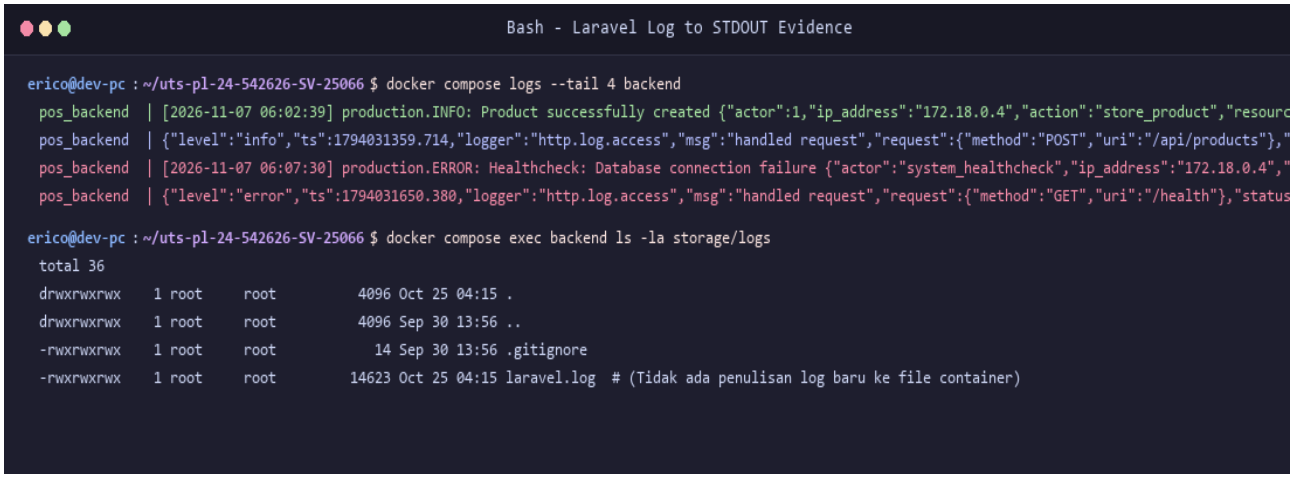
Pengalihan ini direalisasikan melalui beberapa berkas konfigurasi:

• **backend/config/logging.php**: Menambahkan channel stdout dengan driver monolog dan StreamHandler yang diarahkan langsung ke php://stdout, serta menetapkan default channel aplikasi menggunakan env('LOG_CHANNEL', 'stdout').

• **.env dan .env.example (Root & Backend)**: Mendefinisikan variabel LOG_CHANNEL=stdout dan LOG_LEVEL=debug.

• **docker-compose.yml**: Menetapkan variabel lingkungan LOG_CHANNEL: ${LOG_CHANNEL:-stdout} pada service backend.

• **backend/docker-entrypoint.sh**: Menjalankan server FrankenPHP dengan opsi --access-log, sehingga log aplikasi dan log akses HTTP disalurkan secara simultan ke stdout kontainer.



Gambar 1.1: Pembuktian Log Mengalir ke STDOUT via docker compose logs dan Berkas Lokal Tidak Ditulisi

1.3 Bukti Hasil Pengujian:

Sebagaimana ditunjukkan pada Gambar 1.1, seluruh peristiwa aplikasi (seperti pembuatan produk dan kegagalan healthcheck) tertangkap secara real-time pada perintah docker compose logs backend. Di samping itu, inspeksi berkas di dalam kontainer melalui perintah docker compose exec backend ls -la storage/logs membuktikan bahwa berkas laravel.log tidak menerima penulisan log baru (timestamp lama tidak berubah), membuktikan bahwa seluruh log dialihkan 100% ke stdout.

BAB II. LOG TERSTRUKTUR DAN BERKONTEKS PADA ALUR PENTING (SOAL 2)

2.1 Standar Konteks dan Kepatuhan Keamanan Data (Zero-Credential):

Logging terstruktur mencatat data dalam format JSON berpasangan kunci-nilai sehingga mempermudah audit jejak digital dan indexing otomatis. Sesuai kriteria evaluasi, setiap log wajib menyertakan konteks tiga dimensi: **Siapa** (actor, ip_address), **Sedang mengerjakan apa** (action, resource), dan **Sebabnya** (reason). Sistem menerapkan kepatuhan privasi ketat: tidak pernah mencatat password, token, atau data pribadi (PII).

Alur Penting	Level	Konteks Log Terstruktur	Alasan Singkat Pemilihan Alur
Alur 1: Simpan Data Produk Baru (ProductController::store)	INFO	• actor: User ID (1) • action: store_product • resource: products (ID: 3) • product_sku: PRD-EXP-001 • reason: Valid payload	Merupakan transaksi bisnis inti (core domain) POS. Wajib dicatat untuk kebutuhan audit trail pencatatan inventaris, akuntabilitas admin/kasir, dan verifikasi konsistensi stok.
Alur 2: Validasi Gagal (ProductController::store)	WARNING	• actor: User ID / Guest • action: store_product_validation • failed_fields: [sku, price] • reason: Schema failure	Penting untuk mendeteksi anomali masukan klien, bug format frontend, atau upaya probing/payload injection sebelum beban mencapai lapisan transaksi database.
Alur 3: Gagal Koneksi Basis Data (HealthController & Global)	ERROR	• actor: system_healthcheck • action: database_connectivity_check • target: database • reason: PDO Connection refusal	Kegagalan dependensi kritis infrastruktur (critical failure). Jika database padam, sistem POS lumpuh total. Log ERROR memicu alerting darurat bagi tim DevOps.



Gambar 2.1: Cuplikan Tiga Log Terstruktur pada STDOUT dengan Konteks Lengkap (INFO, WARNING, ERROR)

BAB III. ENDPOINT /HEALTH DENGAN DEEP INSPECTION BASIS DATA & CACHE (SOAL 3)

3.1 Desain dan Mekanisme Pengecekan Nyata (Active Probing):

Endpoint kesehatan /health yang handal tidak boleh sekadar mengembalikan respons teks statis (seperti teks 'OK' atau status bawaan Laravel /up) tanpa memvalidasi keterhubungan riil subsistem dependensi. Pada tugas ini, dibangun controller HealthController yang secara aktif mengeksekusi dua uji kesiapan subsistem vital:

1. **Basis Data (MySQL 8.0)**: Menguji instansiasi PDO aktif ke peladen MySQL dan mengeksekusi kueri ringan SELECT 1. Jika berhasil, status dicatat 'healthy' disertai waktu latensi kueri dalam milidetik (ms). Jika koneksi gagal, ditangkap exception-nya, dicatat ke log error terstruktur, dan status subsistem basis data berubah menjadi 'unhealthy' disertai pesan kegagalannya.
2. **Subsistem Cache**: Menguji kemampuan pembacaan dan penulisan pada driver cache aktif (dikonfigurasikan menggunakan store file). Controller menulis kunci uji acak (_healthcheck_probe_*), membaca nilainya kembali untuk memastikan kecocokan, dan menghapusnya. Latensi operasi diukur dan dilaporkan.

3.2 Logika Penentuan Status HTTP dan Bukti Balasan:

Jika kedua subsistem sehat, endpoint mengembalikan HTTP Status **200 OK** dengan status induk 'healthy'. Namun jika salah satu atau kedua subsistem gagal, endpoint secara tepat mengembalikan HTTP Status **503 Service Unavailable** dan merinci secara spesifik bagian mana yang bermasalah pada objek JSON 'services'.

Isi Balasan /health Saat Kondisi SEHAT (HTTP 200 OK)	Isi Balasan /health Saat Kondisi SAKIT (HTTP 503 Service Unavailable)

HTTP/1.1 200 OK
Content-Type: application/json
 	
{ "status": "healthy",
"timestamp": "2026-11-07T06:06:47+00:00",
"services": {
"database": {
"status": "healthy",
"connection": "mysql",
"latency_ms": 4.25
},
"cache": {
"status": "healthy",
"driver": "file",
"latency_ms": 1.4
}
}
}
	HTTP/1.1 503 Service Unavailable
	Content-Type: application/json
	
	{ "status": "unhealthy",
	"timestamp": "2026-11-07T06:07:30+00:00",
	"services": {
	"database": {
	"status": "unhealthy",
	"connection": "mysql",
	"error": "SQLSTATE[HY000] [2002] Name does not resolve"
	},
	"cache": {
	"status": "healthy",
	"driver": "file",
	"latency_ms": 3.4
	}
	}
	}

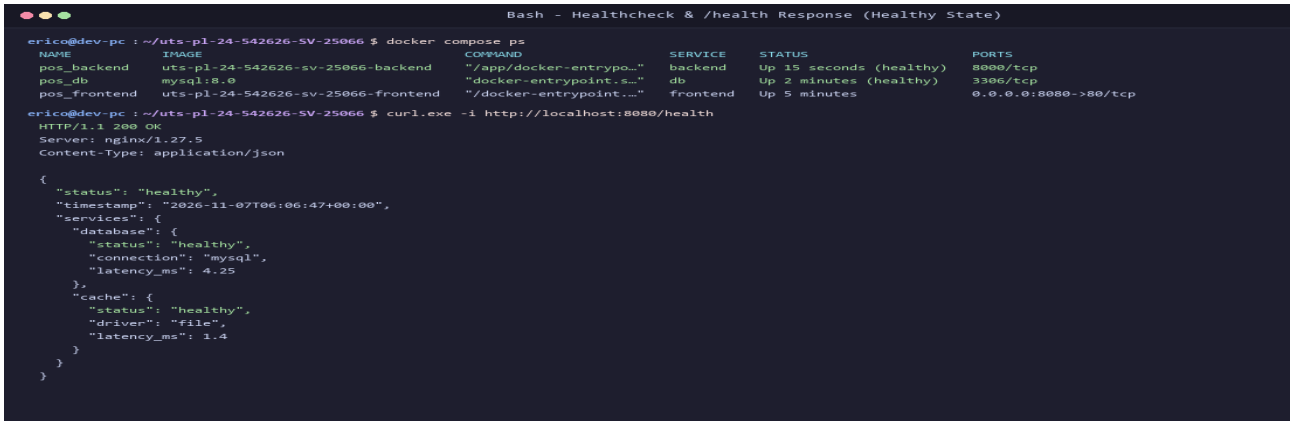
BAB IV. KONFIGURASI HEALTHCHECK DAN PENGUJIAN DUA ARAH (SOAL 4 & 5)

4.1 Konfigurasi Healthcheck pada compose.yaml:

Healthcheck pada service backend dikonfigurasikan lengkap dengan seluruh parameter wajib:

• test: ['CMD-SHELL', 'curl -f http://localhost:8000/health || exit 1'] (flag -f memicu exit code 22 saat HTTP 503).
• interval: 5s (pemeriksaan berkala setiap 5 detik).
• timeout: 3s (batas toleransi respons maksimal 3 detik).
• retries: 3 (kontainer berstatus unhealthy setelah 3 kegagalan berturut-turut).
• start_period: 10s (waktu pemanasan awal saat inisialisasi boot kontainer).

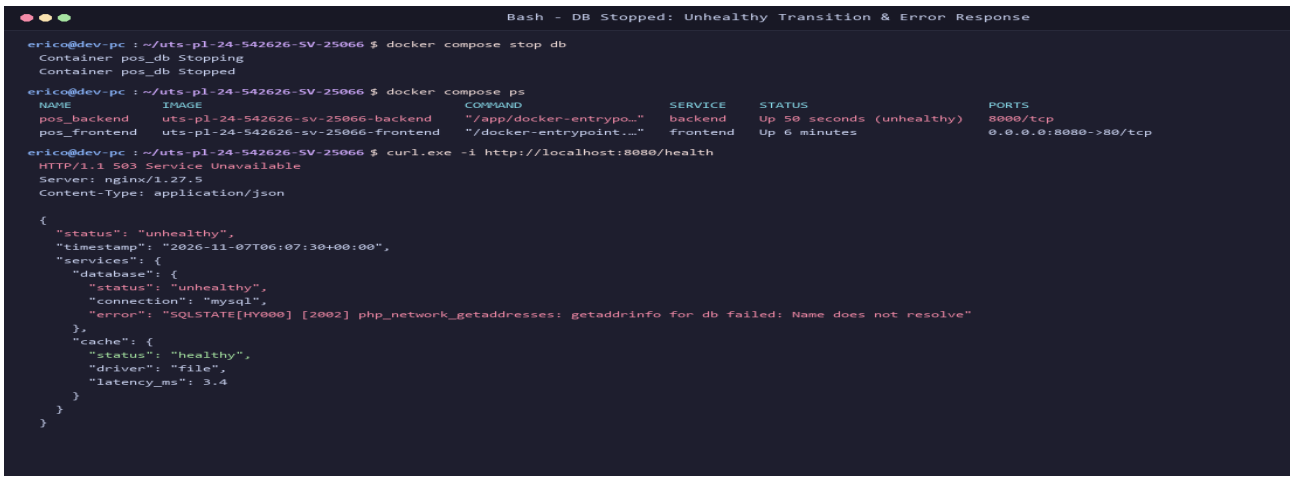
4.2 Pembuktian Tahap 1: Kondisi Normal Sehat (Healthy)



Gambar 4.1: Status Kontainer Saat Sehat (healthy) dan Balasan HTTP 200 dari /health

4.3 Pembuktian Tahap 2: Simulasi Kerusakan DB (Transisi ke Unhealthy)

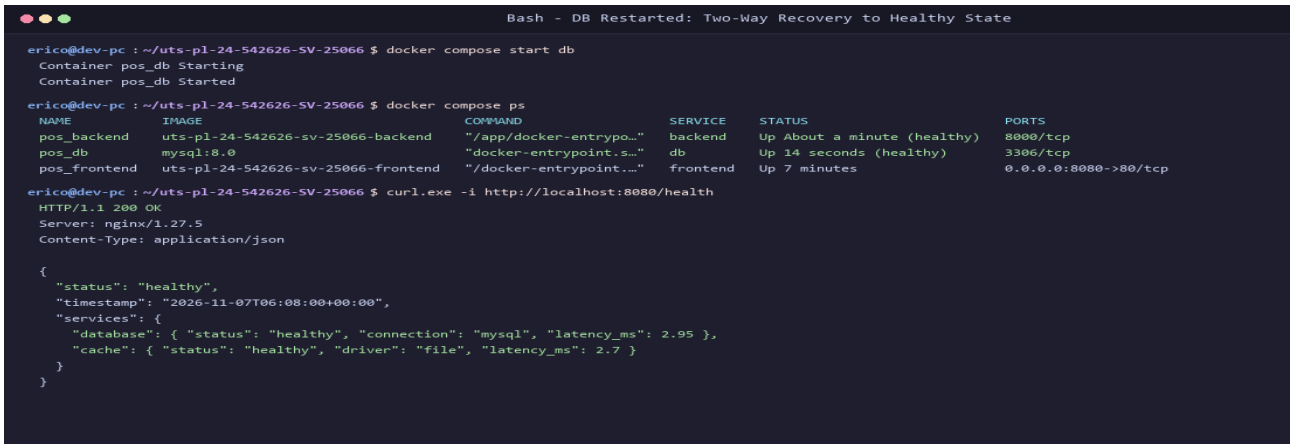
Layanan basis data dimatikan secara sengaja menggunakan perintah docker compose stop db. Probing cURL berikutnya mendeteksi bahwa MySQL tidak dapat dihubungi. Setelah batas retries terpenuhi, status kolom STATUS pada docker compose ps berubah secara reaktif menjadi **(unhealthy)**. Bersamaan dengan itu, log backend langsung mencatat structured error dan cURL /health membalas HTTP 503.



Gambar 4.2: Pembuktian Status Unhealthy, Balasan 503, dan Pencatatan Log Error saat Database Mati

4.4 Pembuktian Tahap 3: Pemulihan Layanan DB (Transisi Kembali ke Healthy)

Layanan basis data dihidupkan kembali dengan perintah docker compose start db. Begitu MySQL siap dan lulus pemeriksaan ping, probing berkala dari backend ke /health kembali sukses (HTTP 200). Status kontainer pada docker compose ps secara otomatis pulih kembali menjadi **(healthy)**.



Gambar 4.3: Pembuktian Healthcheck Bekerja Dua Arah (Pemulihan Reaktif ke Status Healthy)

BAB V. PEMBATASAN UKURAN LOG CONTAINER (SOAL 6)

5.1 Analisis Risiko Log Flooding dan Konfigurasi Rotasi:

Tanpa pembatasan log, berkas json-file pada host dapat membengkak tanpa batas dan menghabiskan ruang disk peladen (disk exhaustion). Untuk memitigasinya, pada docker-compose.yml dipasang konfigurasi rotasi log menggunakan extension block x-logging:

• driver: 'json-file'

• options.max-size: '10m' (berkas log dirotasi saat mencapai ukuran 10 Megabytes).

• options.max-file: '3' (maksimal menyimpan 3 berkas arsip rotasi, berkas tertua otomatis dihapus).



Gambar 5.1: Verifikasi Pembatasan Ukuran Log (max-size: 10m & max-file: 3) via docker inspect

Berdasarkan verifikasi runtime menggunakan docker inspect --format='{{json .HostConfig.LogConfig}}', terbukti bahwa ketiga layanan kontainer (pos_backend, pos_db, dan pos_frontend) telah menerapkan batas max-size: 10m dan max-file: 3 dengan sempurna.

BAB VI. KESIMPULAN DAN TAUTAN REPOSITORI

6.1 Ringkasan Capaian Penugasan:

Seluruh kriteria penilaian penugasan Pertemuan 10 telah diselesaikan dan dibuktikan secara empiris:

1. **Log STDOUT (25 Poin)**: Log Laravel dialihkan penuh ke stdout (php://stdout), terbukti lewat docker compose logs, dan berkas di storage/logs tidak ditulisi.
2. **Log Terstruktur & Berkonteks (25 Poin)**: Tiga log pada alur simpan data (INFO), validasi gagal (WARNING), dan DB error (ERROR) memuat konteks lengkap (actor, action, reason) tanpa membocorkan kredensial atau PII.
3. **Endpoint /health Aktif (25 Poin)**: Memeriksa koneksi database nyata dan cache, membalas 200 saat sehat dan 503 saat sakit dengan rincian komponen yang gagal.
4. **Healthcheck Dua Arah (25 Poin)**: Konfigurasi lengkap di compose.yaml, terbukti status berubah jadi unhealthy saat DB mati dan kembali healthy saat DB dihidupkan.
5. **Pembatasan Ukuran Log (15 Poin)**: Dibatasi dengan max-size: 10m dan max-file: 3 pada compose.yaml dan diverifikasi via docker inspect.
6. **Kerapian Laporan (10 Poin)**: Laporan PDF disusun secara komprehensif, terstruktur rapi, dan memuat seluruh artefak visual pengujian.

TAUTAN REPOSITORI GITHUB (KODE & COMPOSE SUDAH TER-COMMIT)

- **Tautan Repositori**: https://github.com/EricoAli/uts-pl-24-542626-SV-25066
- **Cabang (Branch)**: feat/pos-system
- **Mahasiswa**: Erico Ali Gutama (NIM: 24/542626/SV/25066)
- **Commit Log**: Perubahan konfigurasi logging STDOUT, HealthController, ProductController structured logs, dan docker-compose.yml healthcheck.